

P3669R1

Non-Blocking Support for `std::execution`

Published Proposal, 2025-05-17

This version:

<http://wg21.link/P3669R1>

Author:

[Detlef Vollmann](#)

Audience:

SG1, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

`std::execution` as currently specified doesn't provide support for non-blocking operations. This proposal tries to fix this.

Table of Contents

- 1 **Revision History**
- 2 **Introduction**
- 3 **Design**
- 4 **Proposed Wording**
 - 4.1 `try_schedule` Additions
 - 4.1.1 Concurrent Scheduler Concept
 - 4.1.2 `execution::try_schedule`
 - 4.1.3 `run_loop`

4.2 Modifications for Concurrent Queues

References

Informative References

§ 1. Revision History

This paper revises P3669R0 - 2025-04-14 as follows:

- drops *concurrent-op-state* and introduces `try_schedule`

§ 2. Introduction

Some execution environments want to make sure that specific operations are non-blocking. These can not signal an event using the facilities of `std::execution` as currently specified as it doesn't provide operations that are guaranteed to be non-blocking.

This became apparent in the proposal for concurrent queues [C++ Concurrent Queues \(P0260\)](#), which has support for non-blocking usage. But the non-blocking operations got a very weird interface when interfacing with `std::execution` due to the lack of non-blocking signalling to an async waiter.

In `std::execution` the basic operation to signal an event is to schedule a continuation. In practice this generally means that a `start` operation is called on an operation state that comes from a sender provided by a scheduler.

This proposal is independent from [C++ Concurrent Queues \(P0260\)](#) as the problem of signalling an event in a non-blocking way is not specific to concurrent queues.

§ 3. Design

The initial revision of this paper proposed to add a `bool try_start()` operation to the operation states that come from a scheduler and require it to be non-blocking (and to return `false` if it would block).

After feedback from several people this initial design was dropped. Instead a new CPO `try_schedule()` is proposed.

If a `start()` on the operation state of a sender returned by `try_schedule()` would block (e.g. for inserting the operation into a work queue protected by a mutex), it immediately calls `set_error(would_block_t())` on the connected receiver.

Unfortunately it is not possible for all kind of schedulers to provide `try_schedule()`. Schedulers that are not backed by an execution context that maintains a work queue but run the operation right away are an example. Schedulers that enqueue the work on a different system are another example.

For this reason, this paper proposes a new concept `concurrent_scheduler`. `concurrent_scheduler` requires the `try_schedule` operation.

While this proposal is generally independent from *C++ Concurrent Queues (P0260)*, we still would like to fix the weird interface in that proposal if possible. For this the proposed wording contains a part that modifies the wording for the queue concepts. The idea here is to make it ill-formed if an async operation is called for a queue that also implements `concurrent_queue` and its scheduler does not implement `concurrent_scheduler`.

§ 4. Proposed Wording

(Sorry, the wording is still formally horribly incomplete, but I think substantially it's fairly complete.)

§ 4.1. `try_schedule` Additions

Add to synopsis in namespace `std::execution`:

```
struct concurrent_scheduler_t {};  
  
struct would_block_t {};
```

§ 4.1.1. Concurrent Scheduler Concept

Add to **Schedulers** [exec.sched]:

1. The concept `concurrent_scheduler` defines the requirements of a scheduler type ([`async.ops`]) that provides `try_schedule()`.

```
namespace std::execution {
    template<class S, receiver R>
    concept concurrent_scheduler =
        std::derived_from<typename std::remove_cvref_t<Scheduler>::concurrent_scheduler<S> &&
        requires (S&& s) {
            { try_schedule(std::forward<S>(s)) } -> sender;
        }
}
```

§ 4.1.2. `execution::try_schedule`

Add to **Sender factories** [`exec.factories`]:

1. `try_schedule` obtains a schedule sender ([`exec.async.ops`]) from a scheduler to potentially start an asynchronous operation without blocking ([`defs.block`]).
2. The name `try_schedule` denotes a customization point object. For a subexpression `sch`, the expression `try_schedule(sch)` is expression-equivalent to `sch.try_schedule()`.
3. *Mandates*: The type of `sch.try_schedule()` satisfies `sender`.
4. If the expression `get_completion_scheduler<set_value_t>(get_env(sch.try_schedule())) == sch` is ill-formed or evaluates to `false`, the behavior of calling `try_schedule(sch)` is undefined.
5. The expression `sch.try_schedule()` has the following semantics:
 1. Let `w` be a sender object returned by the expression, and `op` be an operation state obtained from connecting `w` to a receiver `r`.
 2. *Returns*: A sender object `w` that behaves as follows:
 1. `op.start()` starts ([`async.ops`]) the asynchronous operation associated with `op`, if it can be achieved without blocking.
 2. If starting the asynchronous operation would block, it immediately calls `set_error(r, would_block_t())`.

§ 4.1.3. `run_loop`

1. *run-loop-scheduler* models `concurrent_scheduler`.

§ 4.2. Modifications for Concurrent Queues

Remove `conqueue_errc::busy_async` from the enumeration and the respective return statements from `try_emplace`, `try_push` and `try_pop` in the *concurrent-queue* concept.

In the *async-concurrent-queue* concept add another bullet to the senders returned by `async_emplace` and `async_pop` after bullet 6:

7. If the scheduler of `r` does not model `concurrent_scheduler`, the program is ill-formed.

§ References

§ Informative References

[P0260]

C++ Concurrent Queues (P0260). URL: <https://wg21.link/P0260>